Algorithmic Thinking

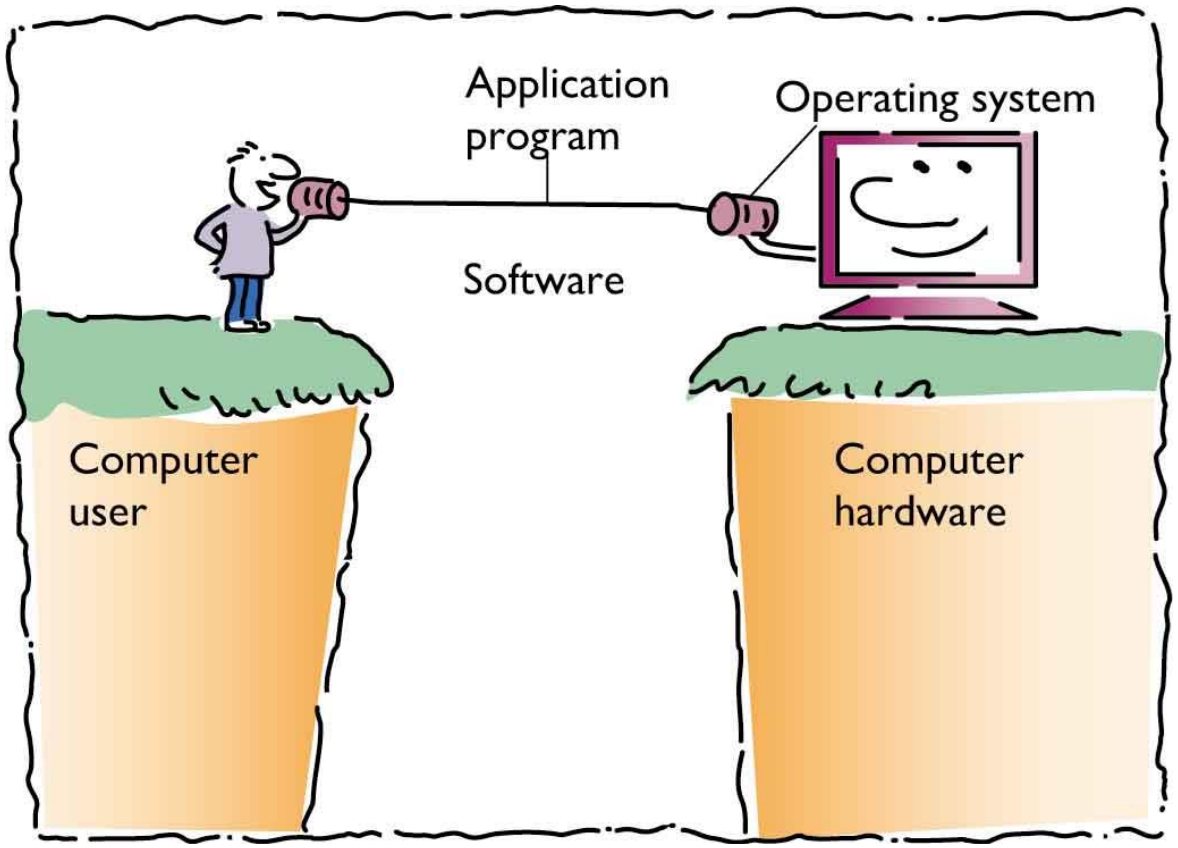
TimPenyusun MateriWI1102Berpikir Komputasional
Institut TeknologiBandung © 2024



Objectives

- Students able to explain concept algorithm as technique computational thinking in solve problem
- Students able to explain concept algorithm(sequential control, selection, repetition, dan control abstraction) to decompose algorithmic to solve simple problem
- Students able to translate concept into simple algorithm in conceptual tool (flow chart/activity diagram/pseudocode)

User – Software – Hardware



Software

- Software allows users to communicate a problem to the computer and the computer provides its solution to the user.
- Without software, a computer is just a dumb machine!

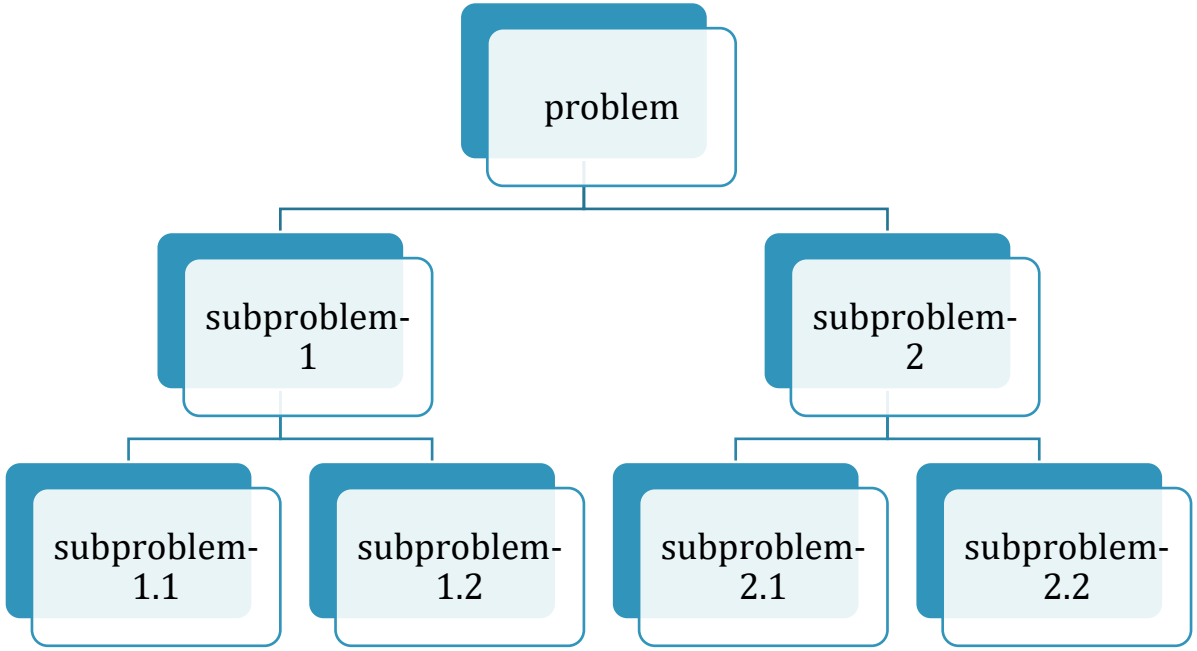
Software = program + data + documentation

Problem Solving with Programming

- **Programming**: is a form of problem-solving where both the problem and its solution are represented in a format that can be processed by a computer.
- In general, it consists of the following steps:
  1. Understanding the problem
  2. Formulating a plan to solve the problem → **algorithm**
  3. Developing a solution based on the plan → **computer program**
  4. Evaluating the solution

From Problem to Program

- Based on the understanding of the problem, the programmer compiles a list of problems and decomposes them into sub-problems.
- Each sub-problem has the potential to become a program module.
- In this course, program modules will be arranged as subprograms.
- **Top down design:** Start from the big problem, then detail it until it finally becomes step-by-step solutions for the sub-problems → algorithm.

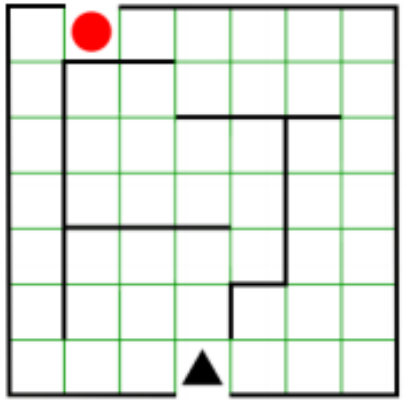


From Ideas to Algorithm

- Algorithm: a set of step-by-step procedures to solve a [sub] problem.
- Can be written using text or diagrams:
  - Pseudocode → text; a cross between human language and programming language
  - Flowchart → diagram
- Algorithms are built using control structures:
  - **Sequence:** steps executed in order
  - **Conditional (branching):** choice of steps
  - **Repetition/loop:** repeating steps

Example-1: Exiting the Maze

Mira needs to find her way out of a maze and asks you to give her directions. She enters the maze from the bottom (black triangle) and must reach the exit at the top (large red circle).



However, Mira can only remember the following four moves:

Kode Gerakan	Artinya	Ilustrasi
A	Berjalan satu langkah maju dan menghadap ke kiri	
B	Berjalan satu langkah maju dan menghadap ke kanan	
C	Berjalan dua langkah maju dan menghadap ke kiri	
D	Berjalan dua langkah maju dan menghadap ke kanan	

Mira can compose a long movement made up of the four moves A, B, C, or D. A move can be repeated several times. This sequence of moves can bring Mira out of the maze with two repetitions.

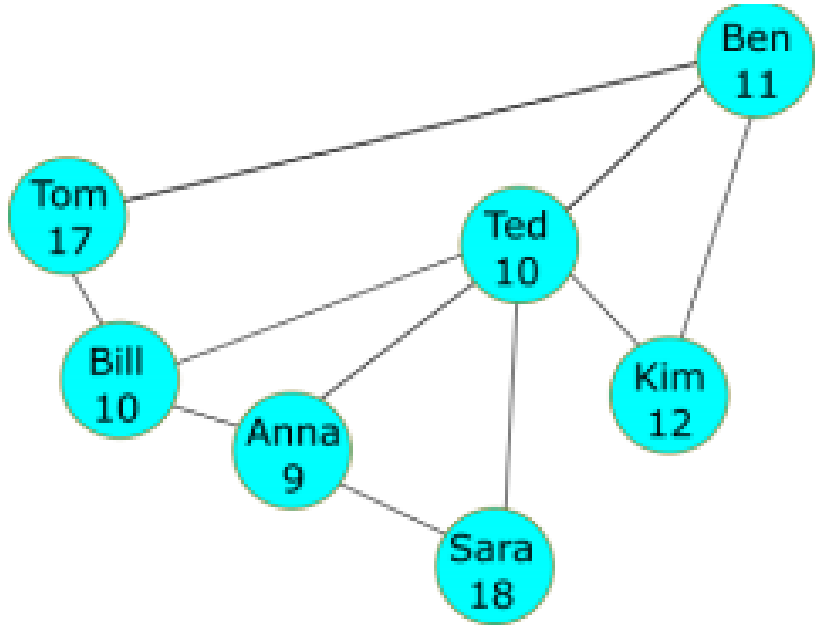
Challenge:
Which sequence of moves, when repeated twice by Mira, will get her to the exit?

Source: Bebras Indonesia (www.bebras.or.id). Problem Code: I-2017-CH-01A

Example-2: Book sharing club

There are seven (7) students who love reading and they form a club to share books. When a student obtains a new book (and has read it), they will then lend it to other club members as follows. Not every student is friends with every other student, so a student only lends the book to those who are friends with them. If a student has multiple friends, the youngest friend who has not yet borrowed the book will receive it first. If all of their friends have already borrowed it, then they return the book to the student who previously lent it to them.

The diagram on the side shows the seven students, and the lines indicate 'friend' relationships. Each node contains the name and age



Challenge:

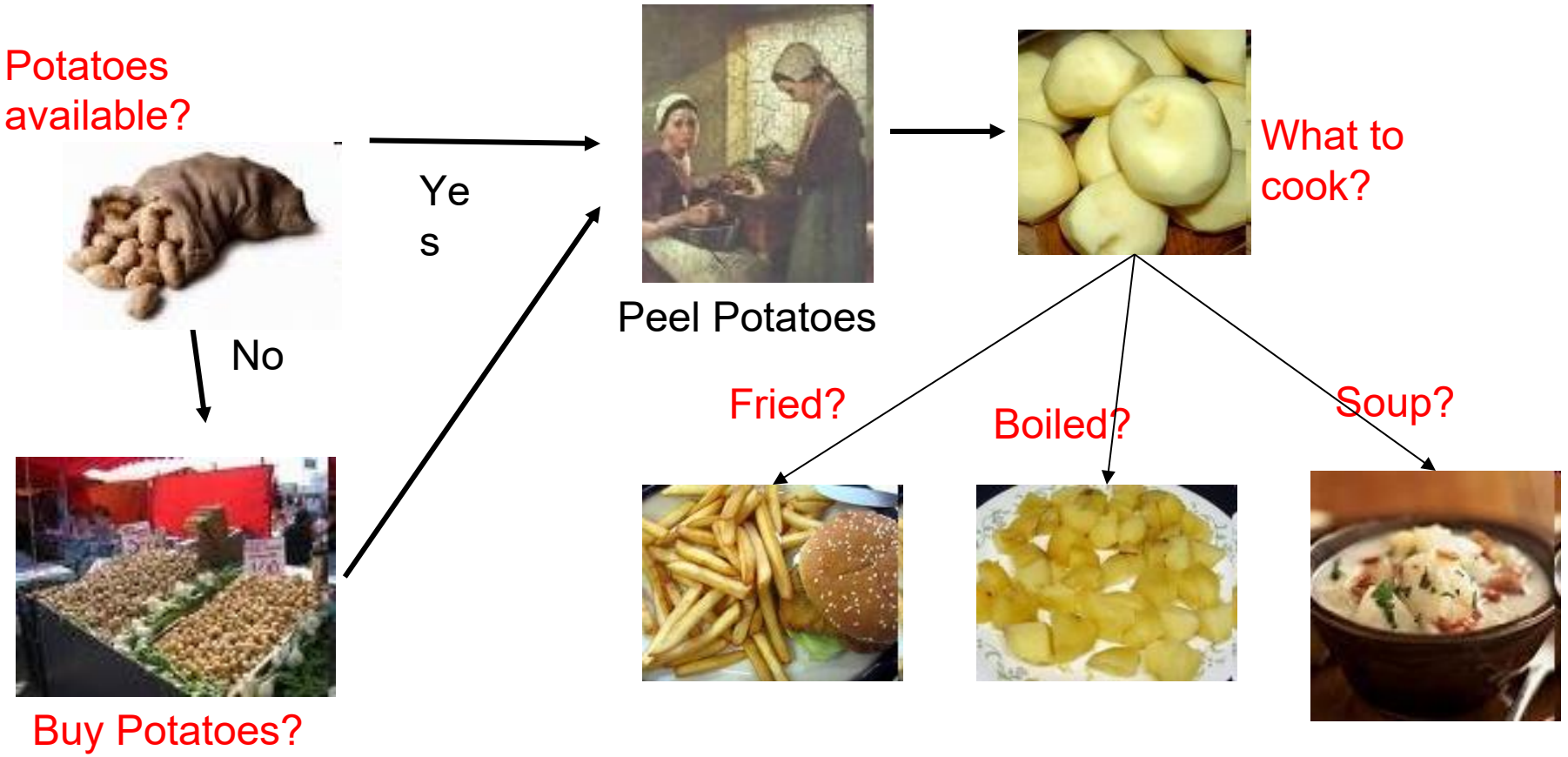
Ben has finished reading a new book and wants to share it with all club members, and apart from Ben, no one has read it yet. Who will be the last person to read the book?

Source: BebrasIndonesia (www.bebras.or.id). Problem Code: I-2017-IR-07

Example-3:
Cooking potatoes for dinner

- “For dinner, a number of potatoes must be peeled and cooked.”
- “Initial state: a bag of potatoes is available in the kitchen.”
  - “It is unclear whether the potatoes are sufficient or not.”
- “Final state: a potato-based dish is available and ready to be served for dinner.”

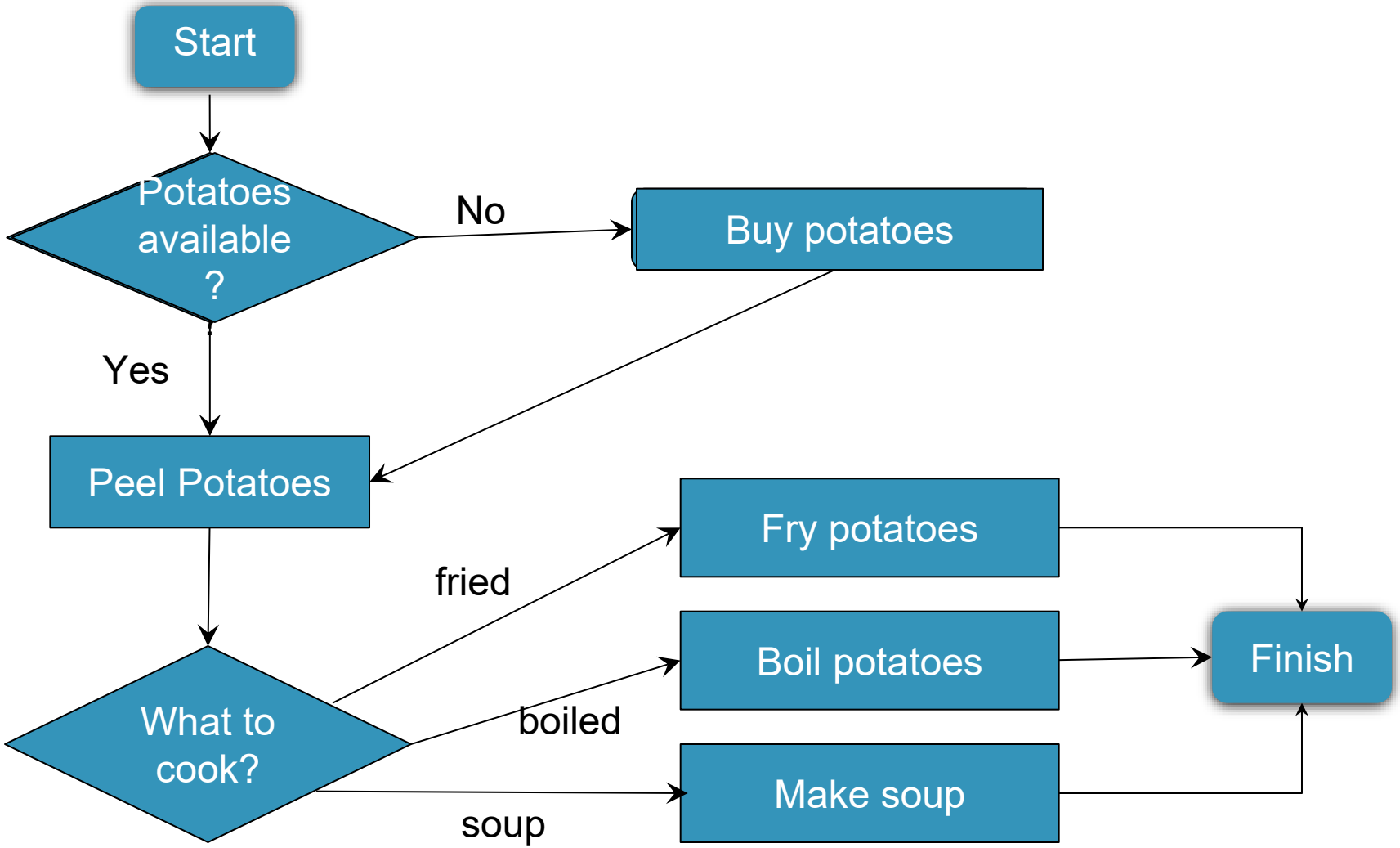
Cooking potatoes for dinner



Cooking potatoes for dinner-Pseudocode

```
if potatoes_available? = no then
      Buy_potatoes
{ At this point potatoes available }
Peel_potatoes
if cooking_selection = fries then
      Fry_potatoes
else if cooking_selection = boiled then
      Boild_potatoes
else{cooking_selection = soup }
      Make_soup
```

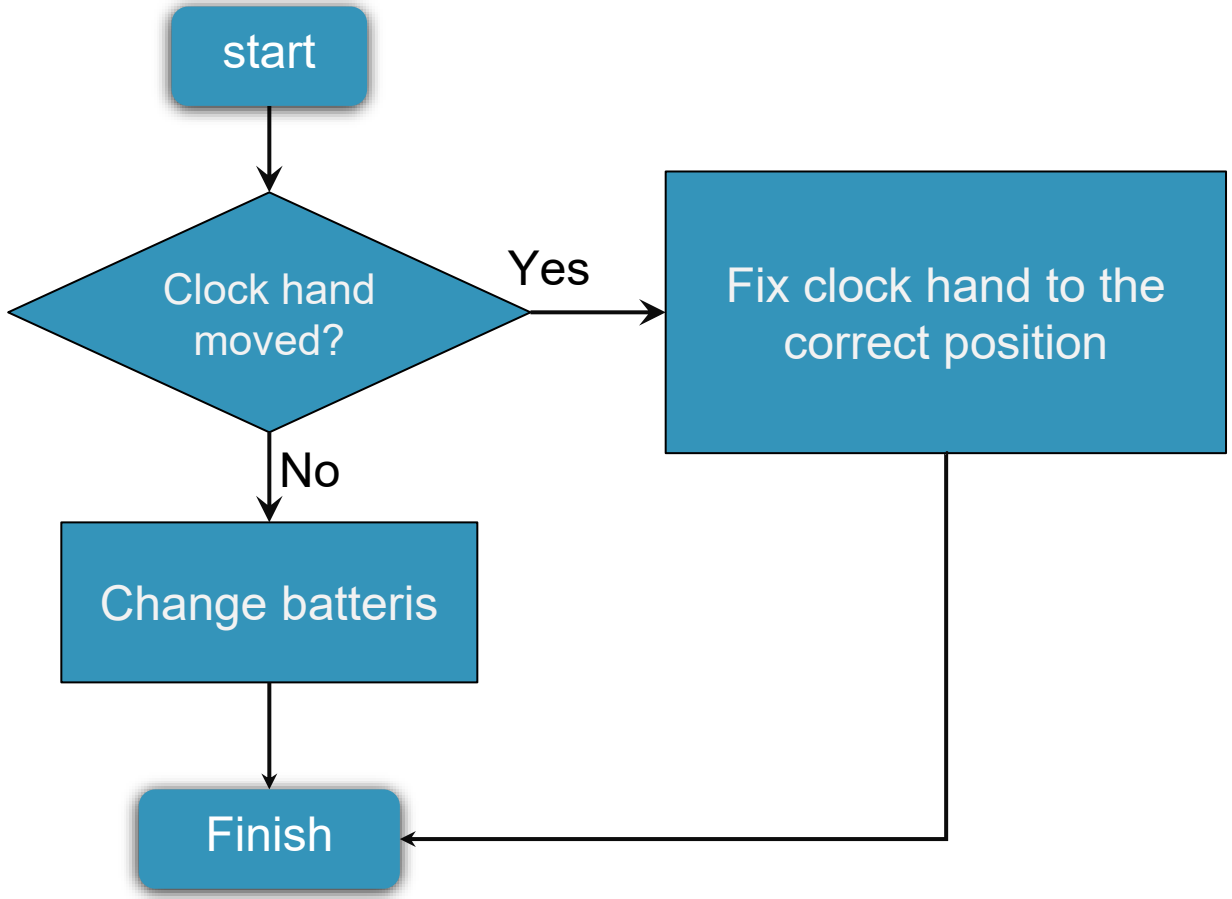
Cooking potatoes for dinner - Flowchart



Example-4:
Fixing wall clock

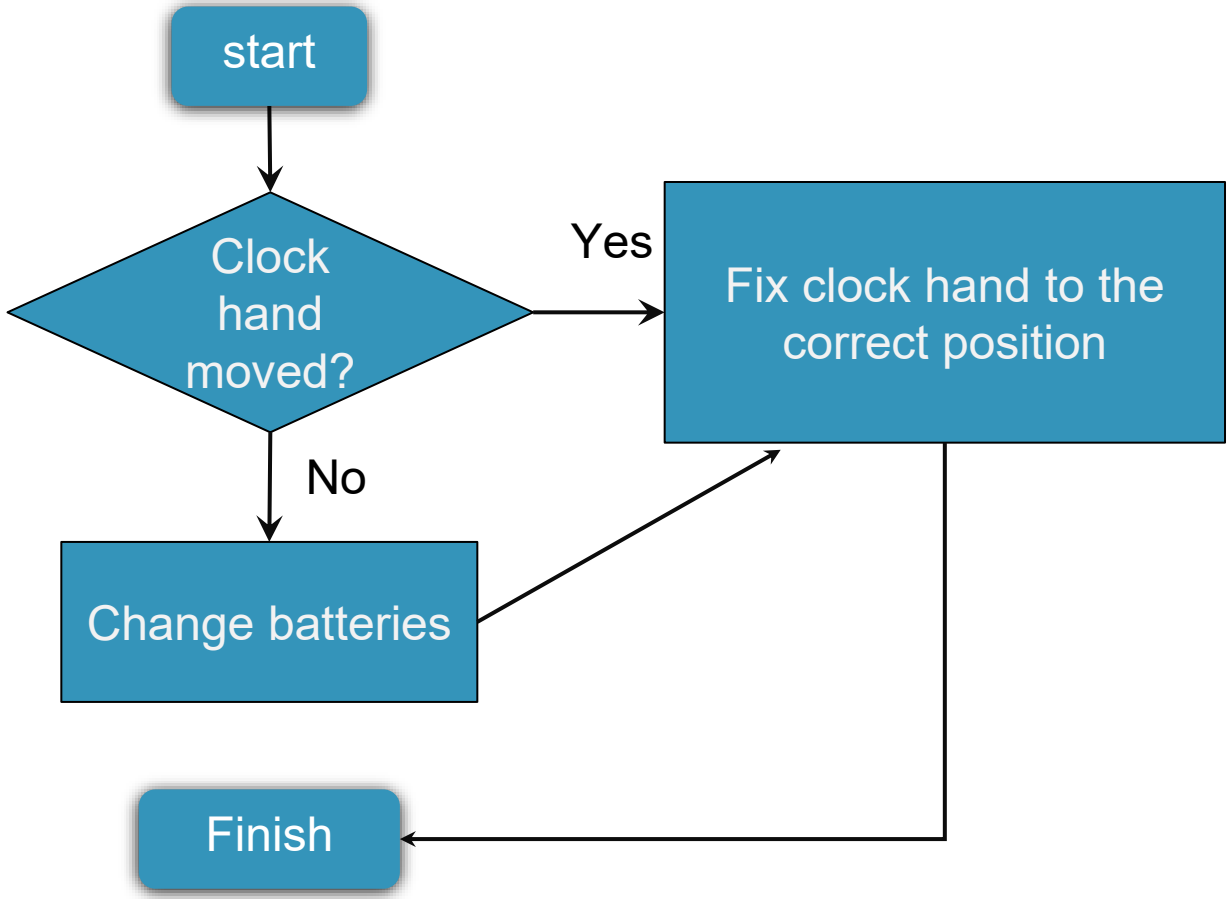
- **Initial state**: The wall clock does not show the correct time.
- **Final state**: The wall clock shows the correct time.
- If the hands do not move, replace the battery.
- If they move, the battery is still alive; simply adjust the positions of the clock hands.

Fixing wall clock: Flowchart-1



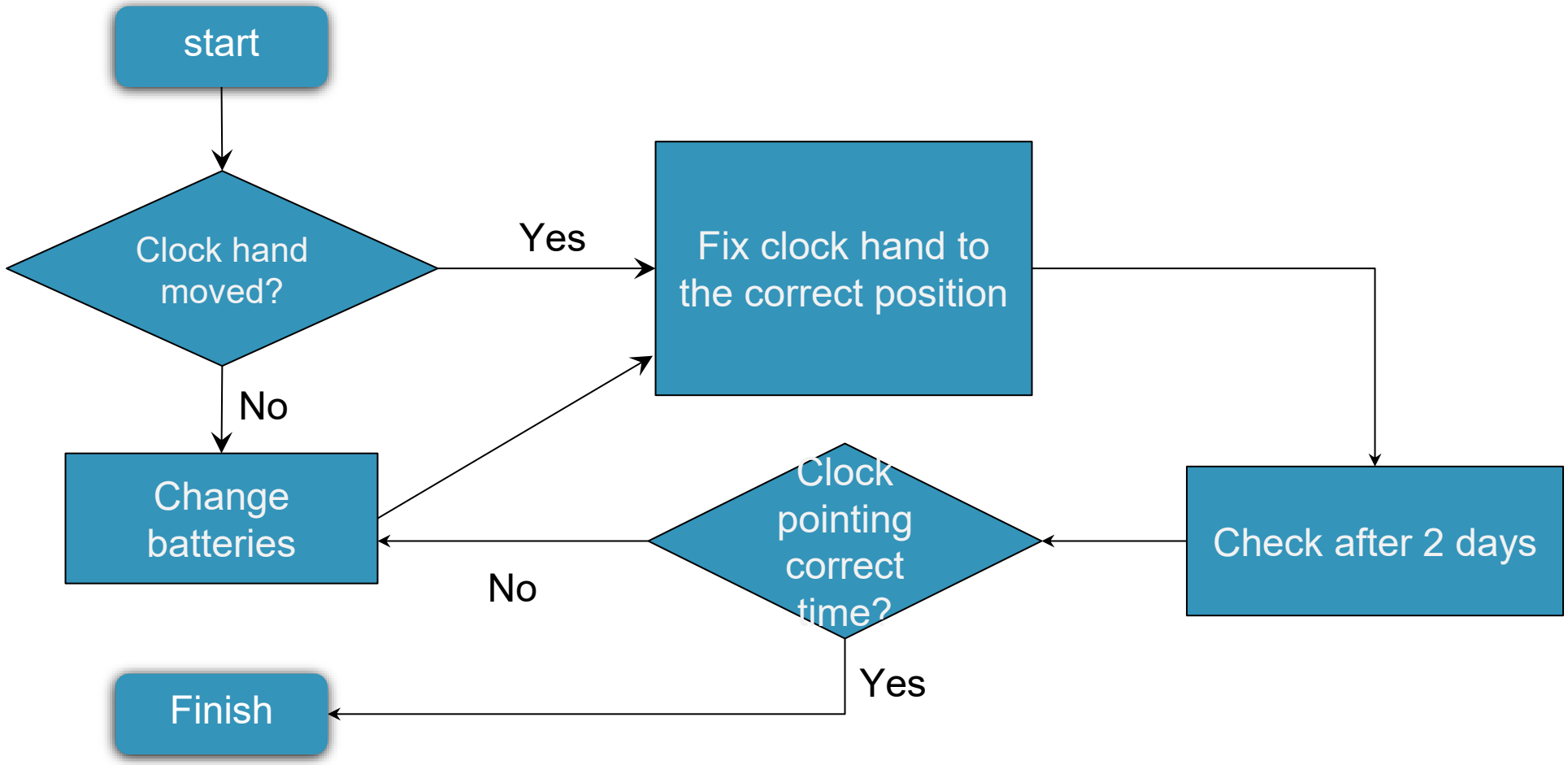
What's wrong with this solution?

Fixing wall clock: Flowchart-2



What if after two days, the clock return to incorrect position?

Fixing wall clock: Flowchart-3



Fixing wall clock: Pseudocode

Refer to Flowchart-3

```
if clockhand_moved? = no then
      Change_batteries
{ At this point clock should moved }
repeat
Fix_position_clock_hand
Check_after_two_days
      if clock_hand_incorrect? = no then
Change_batteries
until (clock_hand_correct? = yes)
```

Example-5
Which photo do you want?

2014-JP-03

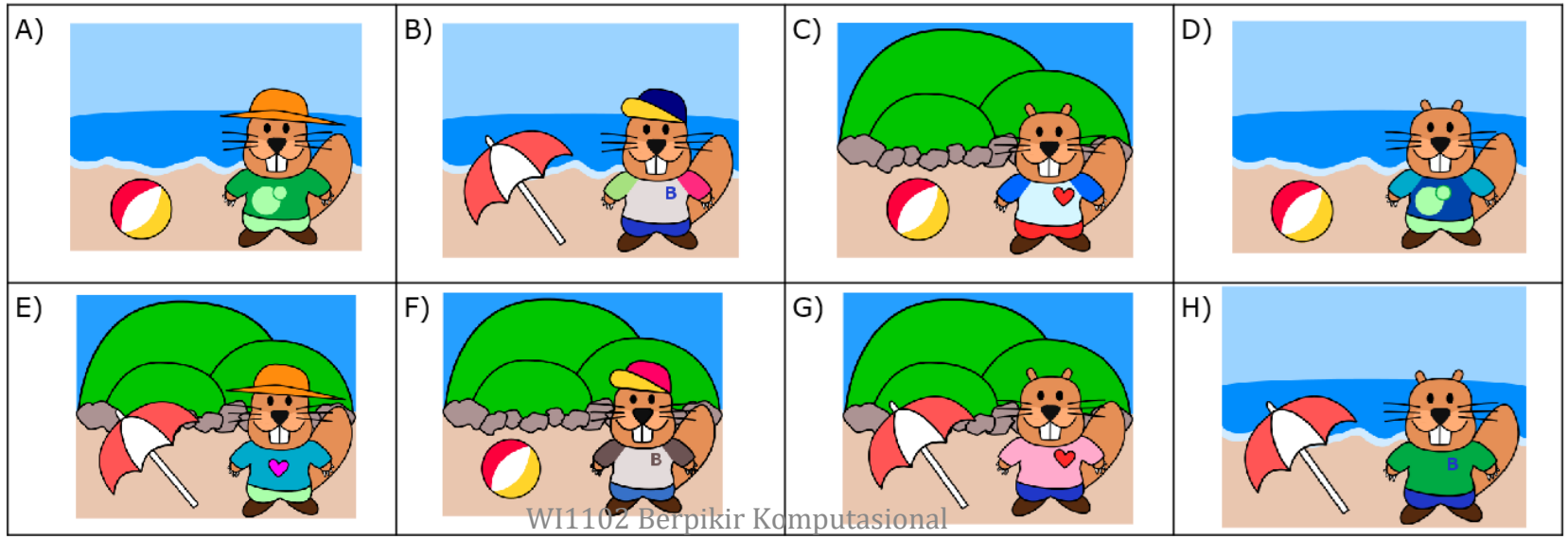
Johnny has 8 photos. He would like to give one of them to Bella. He asks her some questions to find out which photo she wants:

"Do you want a photo with a beach umbrella?" "Yes."

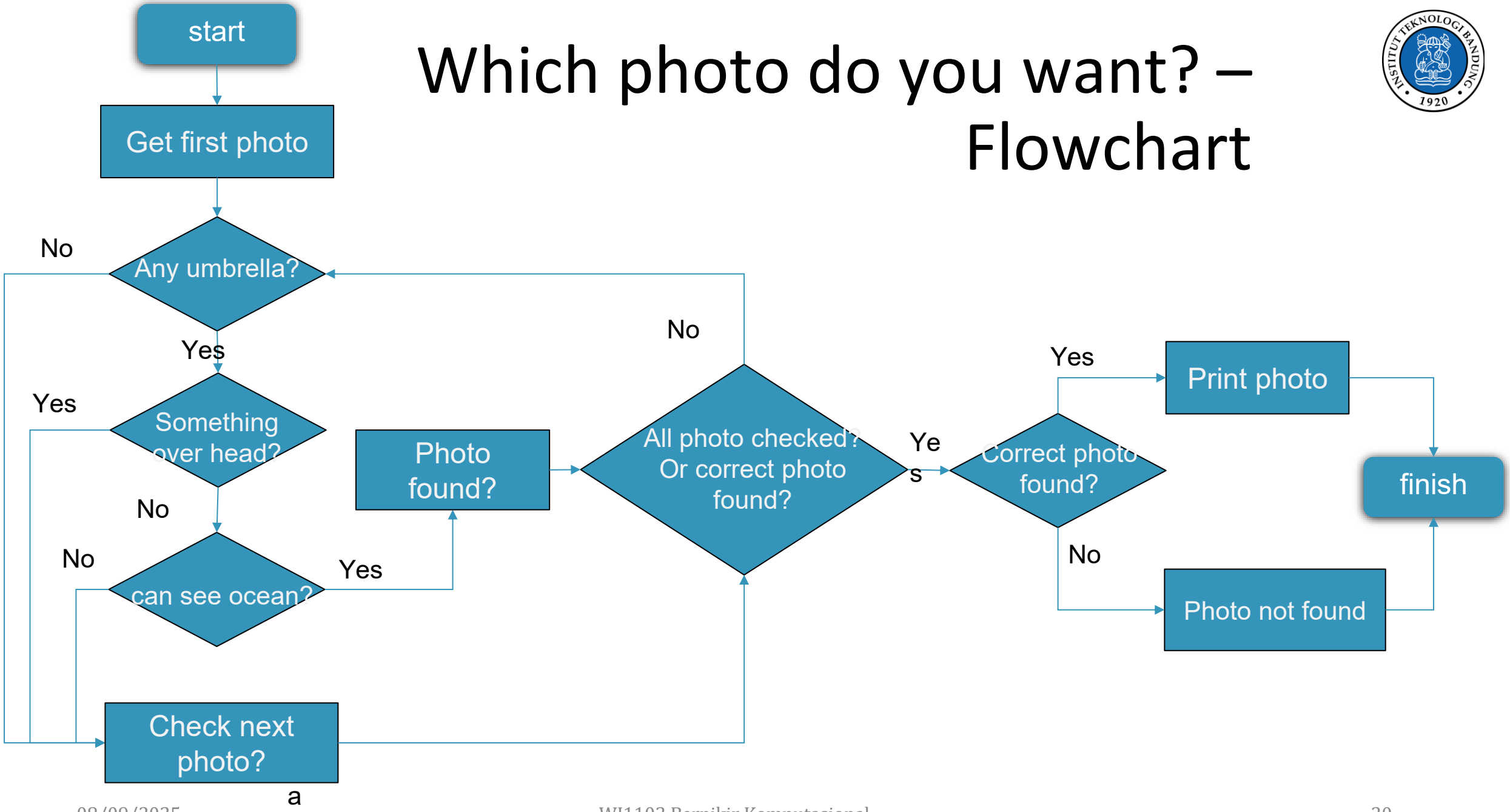
"Do you want a photo where I have something on my head?" "No."

"Do you want a photo where you can see the sea?" "Yes."

Which photo should Johnny give to Bella?(Source: Bebras Challenge)



Which photo do you want? – Flowchart



Which photo do you want? – Pseudocode

```
Take_first_photo
repeat
    if anay_umbrella? =yes then
        if something_on_head? =no then
          if can_see_ocean? =yes then
              photo_found
          else
              check_next_photo
      else
          check_next_photo
    else
          check_next_photo
until(all_photos_had_been_checked) or
(correct_photo_found)

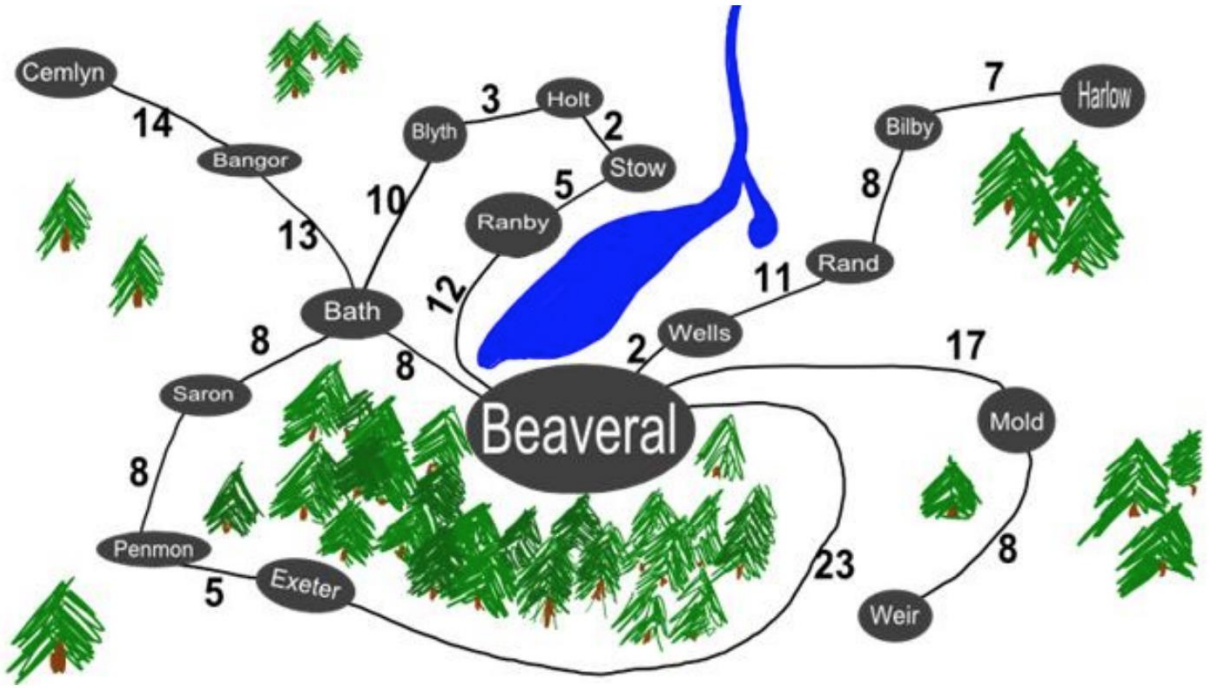
if(correct_photo_found)then
    print_photo
else
    photo_not_found
```

Example-6: Invitation Pickups

Patra lives in the village of Beaveral. He wants to invite his friends to celebrate his birthday. However, he only invites friends who live no more than 20 km from his house so they won't get home too late. The distances to his friends' houses are shown as numbers on the map below.

Challenge:
For invited friends whose homes are more than 17 km away, transportation will be provided. Determine which friends Patra will invite and which ones will be picked up.

Additional challenge:
Write the steps you used to obtain the solution in the form of a flowchart/pseudocode. Can you generalize these algorithmic steps so they can be used on other maps?



Source: Bebras Indonesia (www.bebras.or.id). Problem Code: I-2017-IR-07

Introduction to Programming Language

From Algorithm to Program

- Programmers convert algorithms into computer program code using a programming language.
- The process of writing program code from an algorithm is called coding. The resulting file is called source code.
- Each statement in the algorithm is translated in detail into program code.
- A compiler/interpreter then translates the program code in a given programming language into a form the computer can understand.

Programming Languages

- Every computer processes instructions in machine language
  - numeric codes used to perform basic operations such as: adding and subtracting numbers, comparing numbers, moving numbers, and repeating instructions.
  - Programmers use high-level programming languages to write program code (e.g., C/C++, Python, MATLAB, Scratch, etc.).

Compiler vs Interpreter

- **Compiler**: reads the entire program code at once and translates it into code understood by the computer. Examples: C/C++, Pascal, Fortran



  - Example: C/C++, Pascal, Fortran
- **Interpreter**: read line of code one by one



  - Example: MATLAB, Python

Programming Paradigm

- A programming paradigm is a viewpoint for solving problems with computer programs.
  - Example paradigms: procedural (imperative), object-oriented, declarative, etc.
- Procedural (imperative) paradigm: a program is based on structuring information in memory and manipulating the stored information.

Program =Algoritma+Data Structure

Programming Languages

- There are THOUSANDS of programming languages in the world today—including procedural languages.
- It is impossible to learn all of them in a single course.
- Therefore, what we teach is “learning to program” through computational thinking and the procedural programming paradigm.
- Some well-known languages: C/C++, Python, MATLAB, Scratch, etc.